

## **Assignment 2: System call**

### **1. Process Creation and Management with `fork`**

Write a program that creates a child process using `fork`. The child process should print its process ID, and the parent should print its own process ID. Use `getpid()` and `getppid()` to retrieve the process IDs. Print a message indicating whether the current process is the parent or child.

### **2. Program Execution with `exec`**

Write a program where the parent process spawns a child process using `fork`. The child process should use `execvp/execlp` to replace its image with a call to an external program. Ensure the parent waits for the child process to finish.

### **3. Inter-process Communication with `pipe`**

Write a program where the parent process creates a pipe and forks a child process. The parent writes a message (e.g., "Hello from parent!") into the pipe. The child reads the message from the pipe and prints it to the console.

### **4. Process Synchronization with `wait`**

Write a program where the parent process creates two child processes. Each child process performs a simple task (e.g., printing numbers). The parent process waits for both children to finish using `wait` or `waitpid`.

### **5. Redirect Output Using `dup`**

Write a program that: Opens a file named `output.txt` in write mode using `open()`. Uses the `dup` system call to duplicate the file descriptor of `output.txt`. Redirects the standard output (`stdout`) to the duplicated file descriptor. Prints a message (e.g., "This will be written to the file.") using `printf()`. After the redirection, ensure the message is written to the file instead of the terminal.

### **6. Implement Input/Output Redirection Using `dup2`**

Opens two files: `input.txt` in read mode and `output.txt` in write mode. Uses `dup2` to redirect: Standard input (`stdin`) to read from `input.txt`. Standard output (`stdout`) to write to `output.txt`. Reads a line from `stdin` using `fgets()` or `read()` and writes it to `stdout`. Ensure that the program processes data from `input.txt` and writes the processed result to `output.txt`.